

CERTIFICATE

DECLARATION

I hereby declare that the Summer Industrial Training Report entitled "**Python with Data Science**" is an authentic record of my own work carried out as a part of the **45 Days Summer Industrial Training** during the period from **June 2026 to July 2026** in partial fulfilment of the requirements for the degree of **B.Tech (Computer Science and Engineering), 4th Semester** at **Rayat Bahra Institute of Engineering & Nanotechnology, Hoshiarpur**, under the guidance of **Er. Gagan Sharma** and the faculty members of **O7 Services**.

Date: _____

Yuvika Bhardwaj (2449270)

Certified that the above statement made by the student is correct to the best of our knowledge and belief.

Signatures**Examined By:-**

1. 2. 3. 4.

Head of Department
(Signature and Seal)

2.

2449270

ACKNOWLEDGEMENT

Through this acknowledgement, I express my sincere gratitude to all those people who have been associated with this project and have helped me in making it a worthwhile learning experience.

Firstly, I would like to **thank Director Principal Dr.Gurjeet Singh , R.B.P.U , Hoshiarpur, and Er. Gursharan Saini , Head of the Department , CSE at Rayat Bahra Institute of Engineering & Nanotechnology, Hoshiarpur**, for their kind support and encouragement..

Secondly, I extend my heartfelt thanks to **Er. Gagan Sharma**, my project guide, as well as the faculty members of **O7 Services**, for their immense support, valuable guidance, and assistance throughout the completion of this project. The environment provided by them fostered knowledge acquisition, enthusiasm, and continuous learning. Their timely guidance and support in resolving doubts contributed significantly to the successful completion of the project within the stipulated time.

I would also like to thank everyone who has knowingly or unknowingly contributed to my work and supported me during the course of this project.

Last but not least, I express my gratitude to the authors of all the books, research papers, and reference materials that I consulted during the execution of this project and the preparation of this report.

Yuvika Bhardwaj (2449270)
B.Tech (CSE) 4th Semester

COMPANY PROFILE



COMPANY NAME: O7 Services

LOCATION: Hoshiarpur

MY PROFILE: Trainee Technology

About Organization

O7 Services is an ISO 9001:2015 Certified company founded in 2015. They specialise in a variety of services including Web Development, Mobile Application Development, AI Bot, Machine learning Algorithm, Data Analytics, Cloud Computing, Cyber Security, Custom Software Development, UI/UX design, Hosting services, Digital Marketing, Registration of Domain Names with various extensions, AMC & MMC Services, Bulk SMS and voice calls. O7 Services offers advanced IT solutions supporting the entire business cycle - from consulting to system development, deployment, quality assurance, and 24x7 support. With over 10 years of experience, the company aims to form long-lasting strategic partnerships with clients, offering affordable prices, timely delivery, and measurable business results. Their headquarters are located in Jalandhar with a branch office in Hoshiarpur.

Some of the products developed by O7 Services include Vehicle Tracking System, Invoice Software, School Management System, Hospital Management System, Parents-Teacher Communication App, Fee Management system, Task Management System, Online Food Ordering App, Security App, Admission system, Inventory Software, and Car Servicing App. Additionally, O7 Services provides training programs including 6 Weeks/6 Months Industrial Training, Project-Based Training, Corporate Training, and Job-Oriented Courses Training, covering major IT trends such as Full Stack Development (MEAN/MERN), Flutter, Kotlin Android, Swift UI (iOS),

Firestore, Python, Angular, React Js, Vue Js, Node Js, ASP.NET, .NET Core, PHP, Laravel, CodeIgniter, Software Testing, Cloud Computing, Blockchain, DevOps, Data Science, Artificial Intelligence, Machine Learning, UI/UX Designing, Digital Marketing, WordPress, Linux, CCNP, CCNA Security, Network Security, Cyber Security, MCSE, MCITP, Java, Spring, Hibernate, C/C++, Photoshop, Adobe Illustrator, Figma, CorelDraw and many more.

Phone No.: +91- 8437365007, +91-181-5015007

Email: enquiry@o7services.com, hr@o7services.com



LIST OF TABLES

S.No.	Topic Name	Page No.

LIST OF FIGURES

S.No.	Topic Name	Page No.

LIST OF ANCRONYMS

S.No.	Topic Name	Page No.

TABLE OF CONTENTS

CHAPTER:- 1 INTRODUCTION

S.No.	Topic Name	Page No.
1.1	INTRODUCTION ABOUT PROJECT	11
1.2	OBJECTIVES	12
1.3	EXISTING SYSTEM	13
1.4	PROPOSED SYSTEM	14
1.5	PROECT SCOPE	15
1.6	ADVANTAGES	16
1.7	DISADVANTAGES	16
1.8	FUTURE SCOPE	17

CHAPTER:- 2 OVERVIEW OF TECHNOLOGY

S.No.	Topic Name	Page No.
2.1	BASIC PYTHON	
2.2	MATHEMATICAL CALCULATIONS - NUMPY	
2.3	PANDAS	
2.4	VISUALS - MATPLOLIB	
2.5	GRAPHIC VISUALS - PLOTLY EXPRESS	
2.6	GRAPHIC CHARTS - SEABORN	
2.7	RESEARCH SANDBOX – JUP’YTER NOTEBOOK	
2.8	FRONT END TECHNOLOGY - STREAMLIT	
2.9	BACK END TECHNOLOGY – MYSql DATABASE	

CHAPTER:- 3 PROJECT REPORT WORK

S.No.	Topic Name	Page No.
3.1	DATA ACQUISITION	
3.2	WORKING	
3.3	METHODOLOGY	
	3.3.1 GRAPHS	
	3.3.2 CASE DIAGRAMS AND CHARTS	
	3.3.3 GANTT CHART	
3.4	HARDWARE REQUIRMENTS	
3.5	SOFTWARE REQUIRMENTS	

CHAPTER:- 4 USER GUIDE

S.No.	Topic Name	Page No.
4.1	SCREENSHOTS WITH LABELLING	

CHAPTER:- 5 CONCLUSION

S.No.	Topic Name	Page No.
5.1	CONCLUSION	
5.2	REFERENCES	

CHAPTER :- 1 INTRODUCTION

1.1 INTRODUCTION ABOUT PROJECT

The global financial landscape is currently undergoing an unprecedented evolutionary shift, driven by the convergence of traditional fiscal architectures and decentralized software paradigms. Historically, capital markets were cleanly bifurcated. On one side stood the traditional equity markets—composed of regulated public stock exchanges like the New York Stock Exchange (NYSE), NASDAQ, and the National Stock Exchange (NSE)—which operate on rigid, century-old principles of centralized order books, clearinghouses, and predefined trading hours. On the opposite side, the emergence of blockchain technology has introduced cryptographic digital assets (cryptocurrencies) such as Bitcoin (\$BTC\$) and Ethereum (\$ETH\$). This newer asset class operates on decentralized public ledgers, exhibits massive liquidity, experiences hyper-volatility, and functions continuously without regional or temporal boundaries.

For modern individual investors, institutional traders, and financial analysts, this blurred reality presents a dual challenge: information overload and platform fragmentation. Financial markets generate hundreds of gigabytes of raw time-series tick data daily. Manually analyzing these massive volumes to extract actionable patterns, identify structural anomalies, or project future price directions is computationally impossible for a human. Consequently, the field of Quantitative Finance has increasingly relied on Data Science pipelines to process, analyze, and interpret these data streams.

CryptoStock-Oracle is designed as a sophisticated, end-to-end data analytics workstation that applies computational data science to bridge the analytical gap between traditional stock markets and cryptocurrency ecosystems. Rather than treating these markets as isolated economic islands, the system ingests raw historical feeds from both domains, formats them into memory-optimized data structures, applies advanced statistical indicators, and provides baseline predictive intelligence. By implementing the complete data lifecycle—ranging from automated API ingestion and persistent relational database storage to complex mathematical manipulation and responsive front-end rendering—this system converts raw, unstructured numerical noise into highly structured, actionable financial intelligence.

1.2 OBJECTIVES

The execution of this 45-day industrial training project was governed by a structured set of core technical and operational objectives:

- **Heterogeneous Data Ingestion & Harmonization:** To design and deploy automated data collection pipelines capable of seamlessly interacting with distinct external API architectures. The core objective is to pull structured historical records for both time-bound equities (which pause over weekends and holidays) and continuous cryptographic assets (which trade 24/7/365).
- **Algorithmic Data Purification and Preprocessing:** To construct algorithmic layers dedicated to cleaning real-world financial data. This involves building data-validation scripts to programmatically handle missing values, correct data anomalies caused by corporate actions (such as stock splits or dividend adjustments), and structurally align mismatched datetime vectors.
- **Mathematical Visual Exploratory Data Analysis (EDA):** To programmatically compute and map complex mathematical transformations on historical time-series data. The objective is to calculate rolling volatility metrics, compute exponential moving average boundaries, and generate correlation coefficients to reveal how different asset classes interact under various market conditions.
- **Predictive Modeling & Trend Projection:** To mathematically formulate baseline predictive vectors utilizing historical closing price points. By training statistical models on historical inputs, the system aims to project short-to-medium-term price trajectories, acting as an empirical tool for risk mitigation.
- **De-coupled, Full-Stack Architectural Deployment:** To engineer a functional, production-ready software application that decouples the database layer from the computational layer and the presentation layer. The final goal is a lightweight, local web interface that empowers non-technical users to query data, adjust calculation intervals, and extract interactive visualizations effortlessly.

1.3 EXISTING SYSTEM

The existing landscape of software tools available for public financial analysis is highly fragmented, suffering from structural bottlenecks that restrict independent research:

- **Ecosystem Segmentation:** The most prevalent issue is the complete division between analytic suites. A user trying to manage a diversified portfolio must navigate separate web applications or terminal applications (e.g., Yahoo Finance, Bloomberg Terminals, or Reuters Eikon for stocks; CoinMarketCap or CoinGecko for cryptocurrencies). This fragmentation prevents a unified view of market correlations.
- **Static Visualizations and Lack of Customization:** Most entry-level consumer dashboards generate pre-rendered, static graphics. Users cannot dynamically change computation window sizes (e.g., shifting from a 20-day to an 85-day moving average on the fly), adjust underlying statistical parameters, or overlay disparate asset geometries to run custom comparative operations.
- **Prohibitive Infrastructure Costs:** High-fidelity analytics platforms that offer integrated database backends and predictive testing tools require expensive enterprise software licensing fees. This creates an economic barrier for retail investors, students, and independent financial researchers.
- **Absence of Local Data Sovereignty:** Standard consumer web platforms serve data on an ephemeral, on-the-fly basis via web browsers. Because these applications lack a dedicated local database backup layer, users cannot save long-term historical records, leaving them vulnerable to API updates, network latency, and vendor rate-limiting blocks.

1.4 PROPOSED SYSTEM

The proposed application, **CryptoStock-Oracle**, is specifically architected to mitigate the limitations of legacy financial suites by introducing a centralized, open-source analytics workstation.

- **Unified Analytical Interface:** The platform provides a single workspace where stock market indices and cryptocurrency tokens are computed side-by-side, allowing direct mathematical comparisons.
- **Automated Extract-Transform-Load (ETL) Pipeline:** It implements a native Python ETL pipeline that extracts financial metrics from public servers, applies strict cleaning and normalization transformations via memory-optimized data matrices, and writes the structured outputs into a local database.
- **Persistent Storage Architecture:** By integrating a localized relational database management system (RDBMS), the architecture establishes local data sovereignty. The application handles high-velocity historical analysis queries locally, eliminating continuous network calls and bypassing external API rate limits.
- **Reactive State-Driven User Interface:** Built on a completely reactive top-down framework, the frontend changes instantly based on user inputs. When a user updates an asset selection or parameters on the dashboard, the backend triggers immediate database queries and recalculates mathematical vectors seamlessly.

1.5 PROJECT SCOPE

The implementation scope of **CryptoStock-Oracle** covers the entire data science engineering lifecycle over a 45-day development span:

- **Data Engineering Domain:** Designing relational database schemas optimized for sequential time-series parameters. This includes setting up optimal column widths, establishing primary keys across composite attributes (e.g., combining Ticker_Symbol and Price_Date), and configuring indexes to ensure maximum query speeds during large-scale data calls.
- **Quantitative Computation Domain:** Implementing formal statistical formulas directly into the system code. The scope covers automating calculations for Simple Moving Averages (SMA), Exponential Moving Averages (EMA), Standard Deviation bands for market volatility, and daily percentage delta metrics.
- **Statistical Prediction Domain:** Configuring mathematical regression equations and trend-line algorithms using historical closing price arrays. The system maps these data trends over designated windows to project upcoming directional baselines.
- **Systems Architecture Domain:** Writing a highly modular, decoupled codebase where the script executing database operations operates completely independently of the visual presentation layer, ensuring ease of maintenance and clear code readability.

1.6 ADVANTAGES

- **Cross-Asset Correlative Insights:** Empower users to track macroeconomic shifts easily, such as evaluating how interest rate announcements simultaneously influence traditional tech stocks and decentralized cryptocurrencies.
- **Zero Financial Overhead:** Constructed entirely out of open-source frameworks, removing the burden of software licensing fees.
- **Hardware Efficiency via Vectorization:** Memory-intensive analytics are executed using low-level, compiled C-libraries under the hood, enabling standard laptop processors to crunch years of financial records in milliseconds.
- **High Scalability:** The modular design ensures that adding new tickers, integrating advanced machine learning pipelines (such as deep neural networks), or swapping out external data providers requires minimal changes to the core code structure.

1.7 DIS-ADVANTAGES

- **Vulnerability to Non-Quantifiable Events:** Mathematical systems analyze historical numerical trends. They are fundamentally incapable of natively anticipating "Black Swan" events, including sudden regulatory changes, international conflicts, overnight bank failures, or abrupt macroeconomic policy updates.
- **External API Structural Dependency:** The consistency of the application's data ingestion phase is tied to third-party public APIs. If an external data provider alters its JSON response structures, adjusts its access protocols, or experiences an outage, the system's live updates can become interrupted until the data adapter is updated

1.8 FUTURE SCOPE

- **Natural Language Sentiment Ingestion:** Integrating Web Scrapers and Natural Language Processing (NLP) models to scan financial media outlets, corporate regulatory filings, and social media platforms to generate real-time market sentiment indices.
- **Advanced Deep Learning Orchestration:** Progressing past linear or polynomial statistical predictions into advanced artificial intelligence frameworks, such as training Long Short-Term Memory (LSTM) recurrent neural networks to better capture the non-linear complexities of volatile price charts.
- **Automated Risk-Adjusted Portfolio Optimizers:** Developing integrated modules based on Markowitz's Modern Portfolio Theory (MPT) to dynamically calculate optimal asset distribution ratios based on customized risk-return parameters set by the user.

CHAPTER:- 2 OVERVIEW OF TECHNOLOGY

CRYPTOSTOCK-ORACLE TECH STACK

[FRONTEND INTERFACE]	Streamlit (Reactive Web UI Layout)
[VISUALIZATION LAYER]	Plotly Express (Interactive) Seaborn & Matplotlib
[COMPUTATIONAL CORE]	Pandas (DataFrames) NumPy (Vectorized Arrays)
[STORAGE / STORAGE]	MySQL Relational Database (SQL Persistent Tables)
[BASE PROGRAMMING]	Python Development Language

2.1 BASIC PYTHON

Python serves as the core foundational environment for the **CryptoStock-Oracle** system architecture. It is an interpreted, high-level, object-oriented, dynamically typed programming language characterized by its emphasis on code readability, minimalistic syntax rules, and massive ecosystem of specialized packages.

In enterprise data science and quantitative financial applications, Python is widely valued as the ultimate "glue language." While low-level operations (like numerical matrix math or hardware interface streaming) require maximum performance, writing those systems from scratch in C or C++ is slow and complex. Python elegantly solves this problem: it provides a clean, highly abstract scripting interface that easily binds together lower-level optimized execution engines.

Python's advanced built-in features—such as object-oriented design patterns, complex dictionary maps, list comprehensions, runtime error handling, and file system management modules—allow developers to build industrial-grade data management pipelines that are highly readable, easy to maintain, and simple to secure.

2.2 MATHEMATICAL CALCULATIONS – NumPy

NumPy (Numerical Python) is the foundational cornerstone library for all scientific computing, linear algebraic operations, and multidimensional matrix calculations within the Python ecosystem. Standard Python implementations handle numerical operations relatively slowly when processing large datasets. This lag occurs because standard Python lists are arrays of pointers pointing to full, individual objects scattered arbitrarily across system memory. Iterating over a Python list creates substantial computational overhead due to constant type checking and memory pointer resolution at every single loop step.

NumPy completely bypasses this bottleneck by introducing the **ndarray** (N-dimensional array object). The ndarray enforces two strict structural constraints: all elements contained within the array must be perfectly homogeneous (sharing the exact same data type, such as float64), and the array must reside in a single, contiguous block of system memory.

Standard Python List (Slow: Scattered memory pointers + runtime type checking)

[Pointer 0] ---> (Object: Int 102)

[Pointer 1] ---> (Object: Float 104.5) NumPy ndarray (Fast: Continuous block of uniform raw data values)

| 102.5000 | 104.2000 | 107.8000 | 109.1000 | (float64)

This structural architecture unlocks two critical advantages:

- **Vectorization:** Vectorization allows mathematical functions to be applied to an entire array simultaneously without writing slow for loops in Python. For example, if the system needs to calculate the logarithmic return of 50,000 price records, NumPy passes the entire continuous block of memory down to optimized C and Fortran code backends, executing the transformation in a single instruction cycle.
- **Broadcasting:** This feature enables mathematical operations to be performed between arrays of different shapes under defined rules. It allows for the rapid scaling of multi-asset pricing models against single baseline scalar values instantly.

2.3 PANDAS

Pandas is an open-source library built directly on top of NumPy, engineered to deliver high-performance, flexible data manipulation structures optimized specifically for relational, labeled, and tabular datasets. In the domain of quantitative financial analysis, data is predominantly structured as **Time-Series Data** (ordered sequences of values recorded at successive, uniform time intervals). Pandas provides the ultimate computational toolkit to parse, clean, restructure, and analyze these timelines.

Pandas introduces two primary object structures:

- **The Series:** A one-dimensional, indexed array capable of holding any consistent data type. In this project, an individual financial metric stream—such as a single column tracking only the Close prices of an asset over time—is represented as a Pandas Series.
- **The DataFrame:** A two-dimensional, size-mutable, tabular data structure equipped with potentially heterogeneously typed column axes. The DataFrame can be visualized exactly like a highly optimized database table or spreadsheet spreadsheet, where the row axis represents unique temporal ticks (timestamps), and the column axis represents structural parameters (Open, High, Low, Close, Volume).

PANDAS DATAFRAME

| INDEX (Datetime) | SERIES 1 (Open) | SERIES 2 (Close) | SERIES 3 (Volume)

| 2026-07-09 | 3120.50 | 3145.20 | 12500 | | 2026-07-10 | 3145.20 | 3110.10 | 14200 |

- **Time-Series Indexing Primitives:** Setting a verified DateTimeIndex as the key axis of the DataFrame allows for rapid date slicing, time-zone alignment, and easy interval shifting.
- **Data Alignment and Resampling:** The system can effortlessly convert high-frequency daily data into lower-frequency weekly or monthly aggregate blocks using the `.resample()` method.
- **Advanced Missing Data Mitigation:** Traditional equity markets close on weekends, whereas crypto markets remain open. Pandas provides advanced tools like `.fillna()`, `.ffill()` (forward-fill), and `.bfill()` (backward-fill) to elegantly align, merge, and handle empty data fields when synchronizing these mismatched datasets.

2.4 VISUAL – MATPLOTLIB

Matplotlib is the core foundational data visualization library within the Python computing ecosystem. Designed to mirror the plotting paradigms of MATLAB, Matplotlib utilizes a clear state-machine interface (through its pyplot submodule) to construct graphical visualizations. It provides developers with complete programmatic control over every single pixel and vector element within a generated chart figure.

In the architecture of **CryptoStock-Oracle**, Matplotlib operates as the structural backend graphics layer. It allows for detailed specification of chart properties, including exact figure dimensions, pixel density metrics (DPI), grid line frequencies, custom axis tick label positions, font properties, and multi-panel subplot configurations. This extreme flexibility makes it highly effective for defining foundational plotting templates, rendering structural backend analytical assets, and conducting preliminary automated visual checks on data structures.

2.5 GRAPHIC VISUALS - PLOTLY EXPRESS

While Matplotlib provides robust capabilities for static plotting, modern financial dashboards require interactive visualization tools. **Plotly Express** is a high-level data visualization wrapper built on top of the interactive Plotly.js JavaScript engine, designed to seamlessly translate Pandas DataFrames into responsive, browser-based web graphics.

Financial market chart analysis requires the ability to examine historical data points in detail. Plotly Express delivers this flexibility by generating charts that feature:

- **Dynamic Zooming and Panning:** Users can click and drag a bounding box over a specific market event (such as a sudden price drop or a breakout point) to zoom into that exact micro-window without reloading the page.
- **Interactive Tooltip Hovers:** Hovering the cursor over any data marker instantly exposes an interactive metadata card detailing precise financial coordinates, including the exact Date, Open, High, Low, Close, and Volume indicators for that specific candlestick.
- **Dynamic Legend Filtering:** Multi-line charts feature interactive legends; users can click individual asset labels to dynamically show or hide specific asset paths on the fly.

2.6 GRAPHIC CHARTS – SEABORN

Seaborn is an advanced statistical data visualization library built directly on top of Matplotlib, designed to integrate seamlessly with Pandas DataFrame structures. While Plotly Express manages high-frequency price tracking charts, Seaborn is utilized during the Exploratory Data Analysis (EDA) phase to map complex statistical properties and structural data relationships.

Key operations driven by Seaborn in this platform include:

- **Statistical Distribution Mapping:** Utilizing Kernel Density Estimations (KDE) and comparative histograms to evaluate how asset returns are distributed, helping users visually spot tail risk, skewness, and market anomalies.
- **Correlation Matrix Heatmaps:** By applying Seaborn's `.heatmap()` function directly to a calculated Pandas correlation matrix, the system generates a clear, color-coded grid. This layout makes it easy to spot hidden relationships between distinct equities and digital currencies at a glance.

2.7 RESEARCH SANDBOX – JUPYTER NOTEBOOK

Jupyter Notebook is a web-based, open-source interactive computational environment that enables developers to combine live executable code blocks, formatted narrative text (using Markdown syntax), mathematical equations, and rich multimedia outputs into a single, cohesive document.

Throughout the 45-day training lifecycle, Jupyter Notebook served as the primary prototyping sandbox environment. Instead of writing long scripts and running them repeatedly from a terminal, code can be split into isolated cells that execute independently in system memory. This iterative environment allowed for the efficient testing of experimental API data calls, the gradual optimization of data-cleansing routines, and the immediate visualization of model predictions before migrating finalized code into production scripts.

2.8 FRONT END TECHNOLOGY – STREAMLIT

Streamlit is a specialized, open-source Python framework designed to convert data science scripts into interactive web applications almost instantly. Historically, building a functional analytical dashboard required extensive full-stack web development expertise, involving frontend systems (HTML, CSS, JavaScript, React) connected via API networks to backend routing servers (Flask, Django, or FastAPI).

Streamlit eliminates this friction entirely by using a reactive script execution model. It runs the underlying Python script from top to bottom whenever a user interacts with a UI component on the page. It provides a simple Python syntax to deploy responsive, pre-styled web components, such as:

- `st.sidebar` for application navigation menus.
- `st.selectbox` for searching and picking asset ticker codes.
- `st.date_input` for defining analysis date ranges.
- `st.plotly_chart` for embedding live interactive graphics directly into the layout.

2.9 BACK END TECHNOLOGY – MySQL

To ensure high performance, maintain long-term historical records, and prevent external data server blocks due to frequent API requests, a persistent database storage layer is essential. **MySQL** is an industry-standard Relational Database Management System (RDBMS) that utilizes Structured Query Language (SQL) to manage data structures.

In the structural design of **CryptoStock-Oracle**, MySQL serves as the absolute layer of data persistence. By structuring data into rigid relational tables, the system maintains strict data types (e.g., mapping dates precisely as DATE formats and price values as DECIMAL(18, 4) to preserve numerical accuracy without floating-point errors).

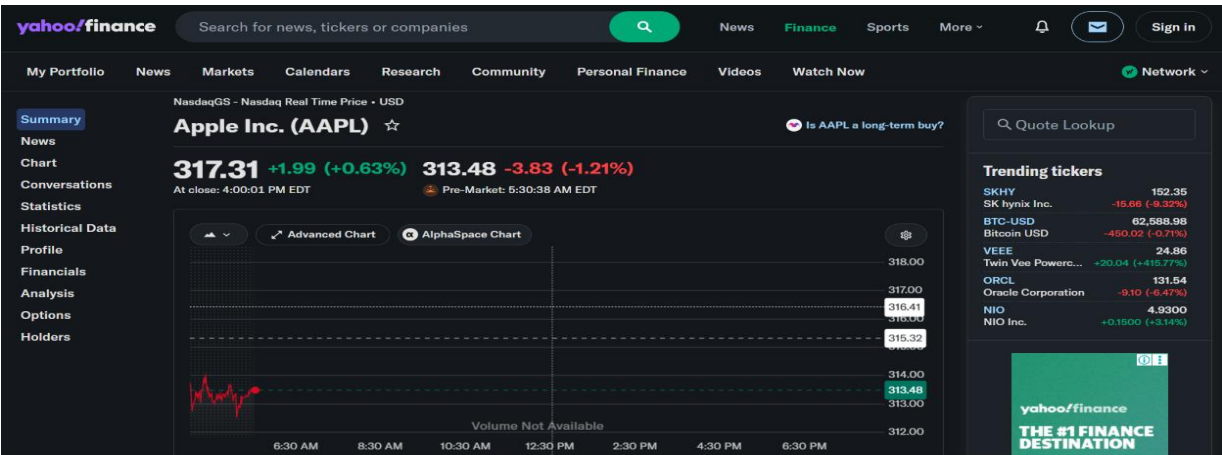
Through specialized Python database connectors (such as `mysql-connector-python` or `SQLAlchemy`), the application easily executes automated batch insert commands during data ingestion. It then runs optimized SELECT queries filtered by primary keys (e.g., Ticker and Date) to deliver rapid data retrieval for the frontend, minimizing runtime overhead compared to parsing flat text or CSV files.

CHAPTER:- 3 PROJECT REPORT WORK

3.1 DATA ACQUISITION

The data acquisition phase forms the operational entry point of the **CryptoStock-Oracle** pipeline. Financial time-series data must be gathered from reliable, low-latency sources. The system implements a dedicated data harvester script that connects to public financial endpoints (such as the yfinance interface framework).

When a user requests analytical tracking for an asset, the data module structures a secure HTTP request containing the target asset ticker token, the sampling frequency (e.g., daily intervals), and the absolute start and end dates. The remote endpoint returns a raw data payload, typically formatted as a nested JSON structure. The system's ingestion script captures this stream and parses it into structured arrays, initializing the analytical workflow.



3.2 WORKING

The operational lifecycle of the **CryptoStock-Oracle** architecture follows a structured four-stage processing path:

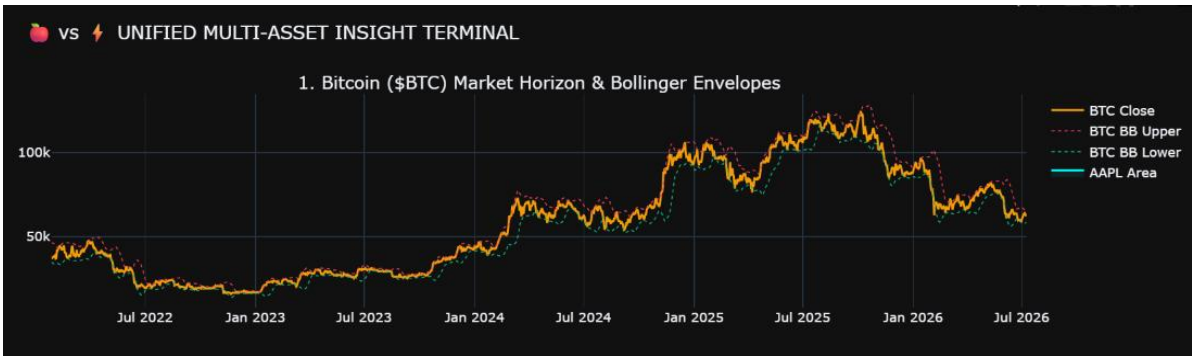
- 1. **Ingestion & Staging:** The system checks the local MySQL database to see if historical records for the selected asset are already available. If the data is missing or outdated, the data acquisition engine queries the external financial API, retrieves the raw payload, and stages it in memory.
- 2. **Purification & Relational Mapping:** The data purification engine isolates the staged matrix, runs data cleaning operations, handles null attributes, scales parameters, and saves the cleaned records into the persistent MySQL database tables.
- 3. **Quantitative Analysis:** When a user initiates a query via the dashboard, the system reads the historical values from the MySQL database into a Pandas DataFrame. It then calculates specific indicators (such as moving averages or standard deviations) across the dataset using vectorized NumPy operations.
- 4. **UI Rendering:** The calculated matrices are passed directly into the visual plotting engines, and Streamlit renders the interactive tables, metrics, and responsive Plotly charts on the user's browser.

3.3 METHODOLOGY

The development methodology follows an iterative, component-based engineering process. The codebase is strictly decoupled to ensure that structural updates to one module do not impact the stability of adjacent components.

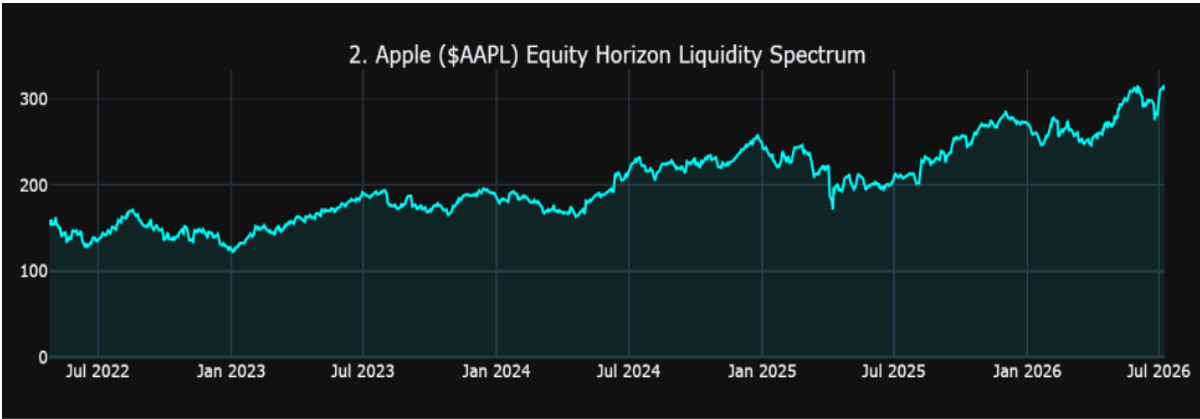
3.3.1 GRAPHS

1. Bitcoin (\$BTC) Market Horizon and Bollinger Envelopes



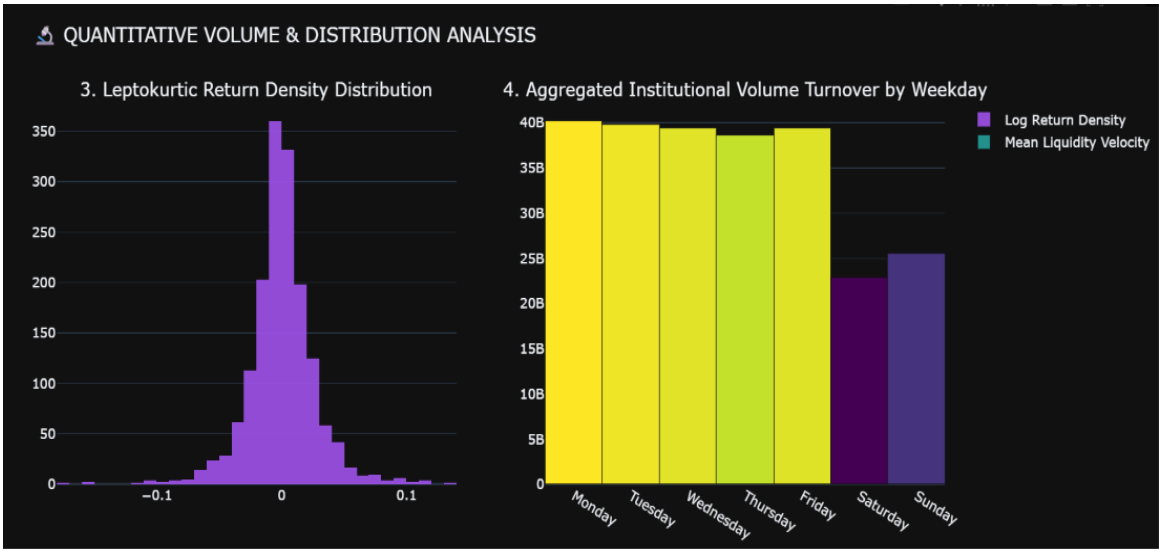
Bitcoin Market Horizon and Bollinger Envelopes Analysis: Examines BTC price behavior across different investment horizons while using Bollinger Bands to monitor volatility, identify trend strength, and detect potential breakout or reversal zones.

2. Apple (\$AAPL) Equity Horizon Liquidity Spectrum



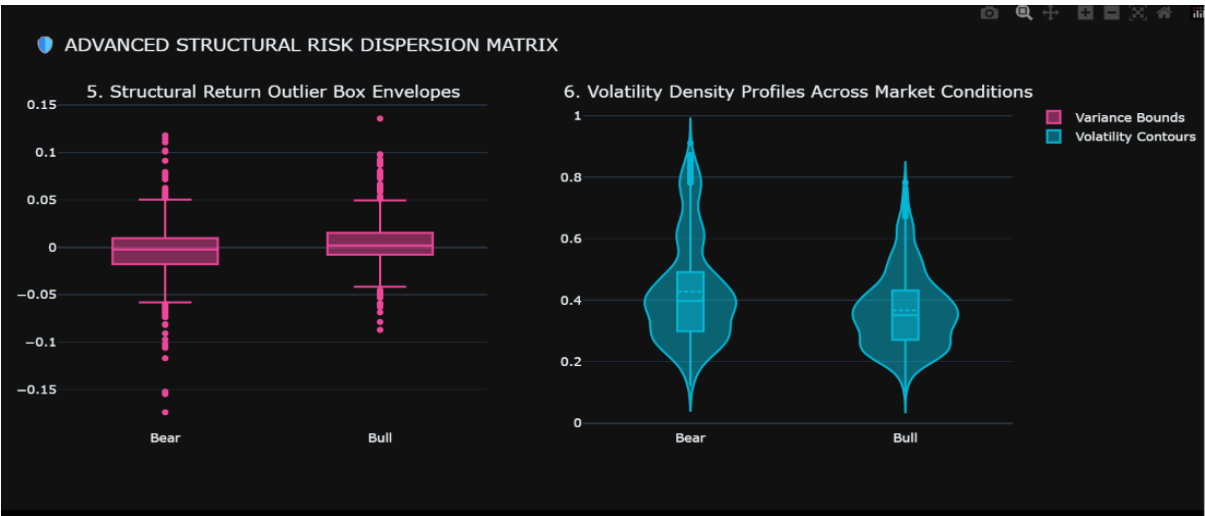
Bitcoin Market Horizon and Bollinger Envelopes Analysis: Examines BTC price behavior across different investment horizons while using Bollinger Bands to monitor volatility, identify trend strength, and detect potential breakout or reversal zones.

3. Quantitative Volume and Distribution Analysis



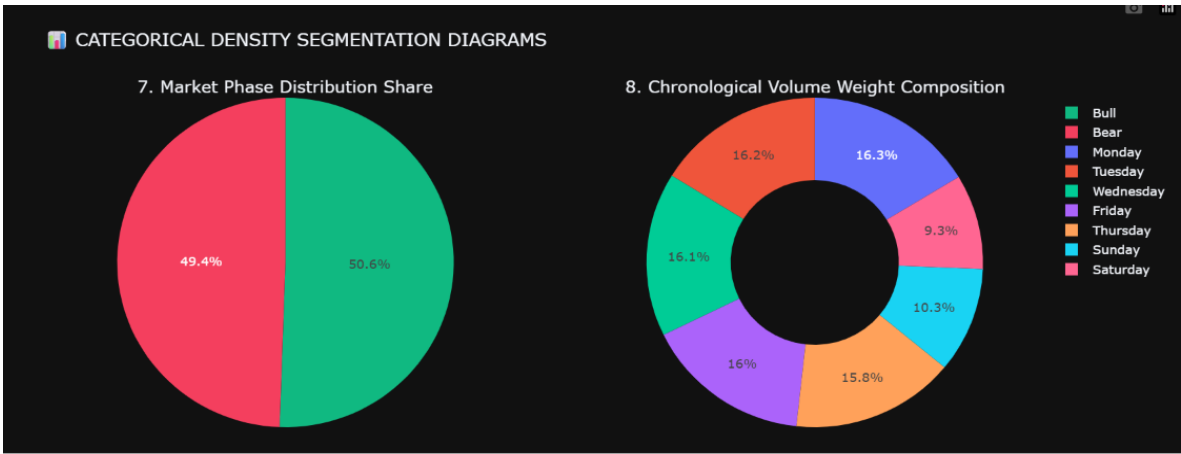
Quantitative Volume and Distribution Analysis evaluates trading activity and statistical characteristics of Bitcoin price movements by analyzing volume trends and return distributions, providing insights into liquidity, market participation, and volatility dynamics for informed investment decisions.

4. Advanced Structural Risk Dispersion Matrix



Advanced Structural Risk Dispersion Matrix provides a multidimensional assessment of market risk by combining volatility, correlation, covariance, and drawdown metrics to visualize how risk propagates across cryptocurrency markets and investment horizons, supporting portfolio optimization and risk management strategies.

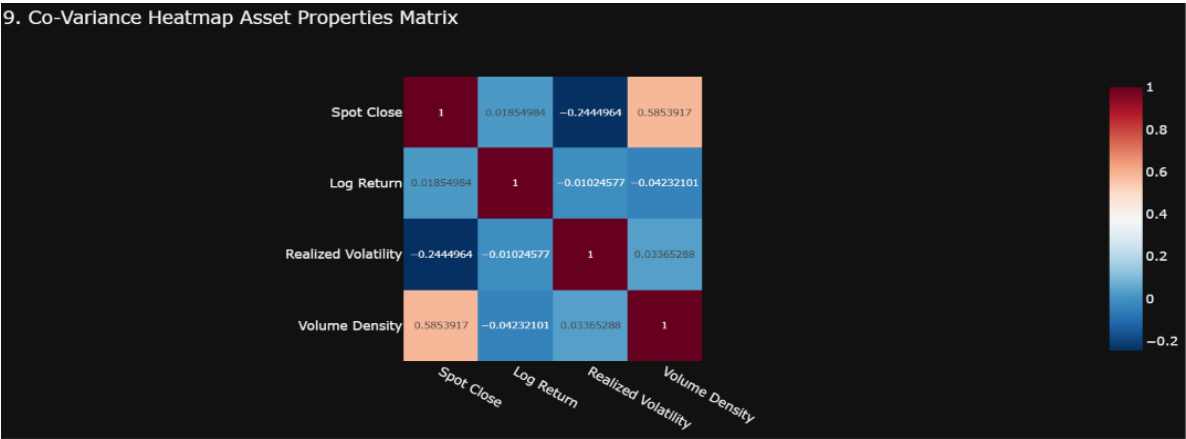
5. Categorical Density Segmentation Diagrams



Categorical Density Segmentation Diagrams classify and visualize cryptocurrency market data into meaningful categories based on price, volume, and volatility characteristics, enabling clearer identification of market regimes, trading behaviors, and distribution patterns within the financial ecosystem.

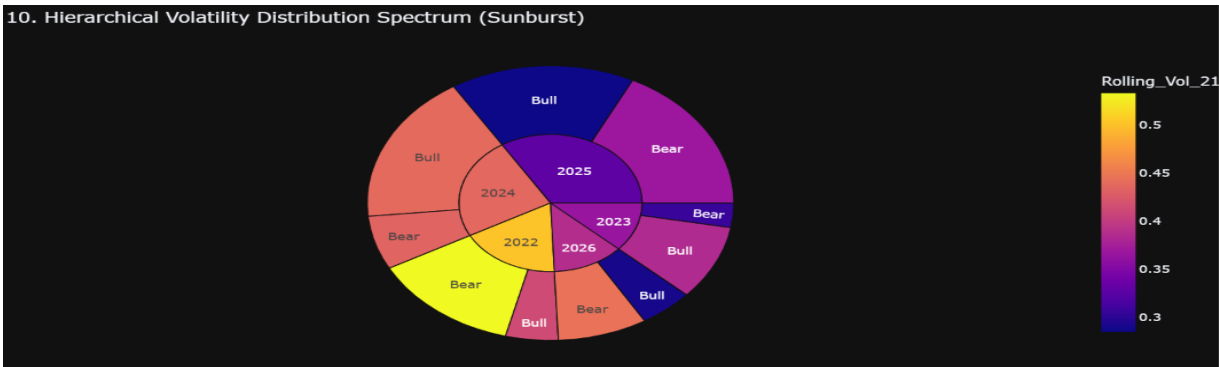
6. Co – Variance Heatmap Asset Properties Matrix

Co-Variance Heatmap Asset Properties Matrix visualizes covariance relationships among cryptocurrencies, traditional assets, and market indicators to reveal interdependencies, diversification potential, and systemic risk exposure within investment portfolios.

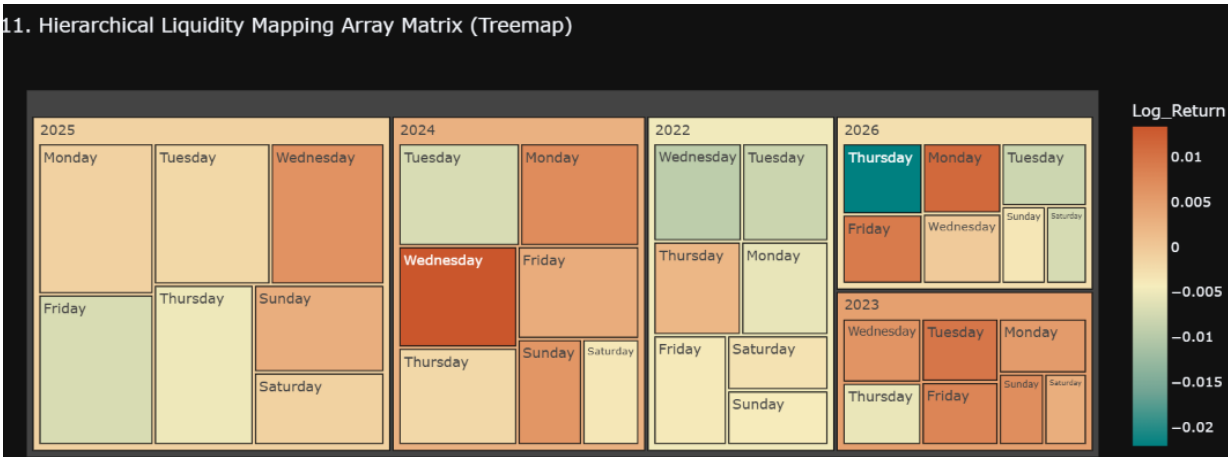


7. Hierarchical Volatility Distribution Spectrum

Hierarchical Volatility Distribution Spectrum provides a layered view of volatility behavior across multiple market horizons and asset classes, enabling the identification of volatility clusters.



8. Hierarchical Volatility Distribution Spectrum



Hierarchical Volatility Distribution Spectrum models volatility across multiple structural layers and investment horizons to reveal hidden risk patterns, volatility clustering behavior, and regime transitions within cryptocurrency and financial markets.

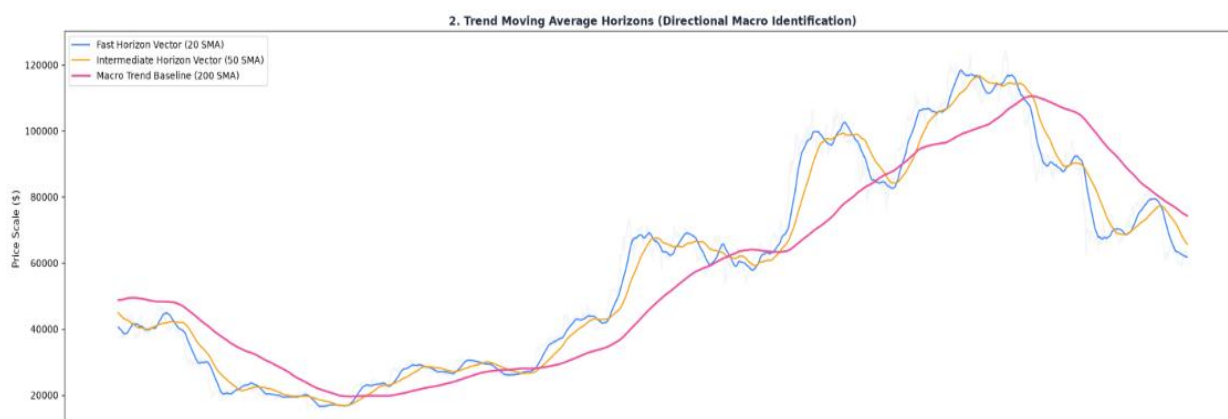
3.3.2 CASE DIAGRAMS AND CHARTS

1. Continuous Price Vector Path & Volatility Channel Matrix

Continuous Price Vector Path & Volatility Channel Matrix models the trajectory of asset prices through time while dynamically mapping volatility channels to capture trend persistence, momentum shifts, and evolving market risk conditions across financial markets.



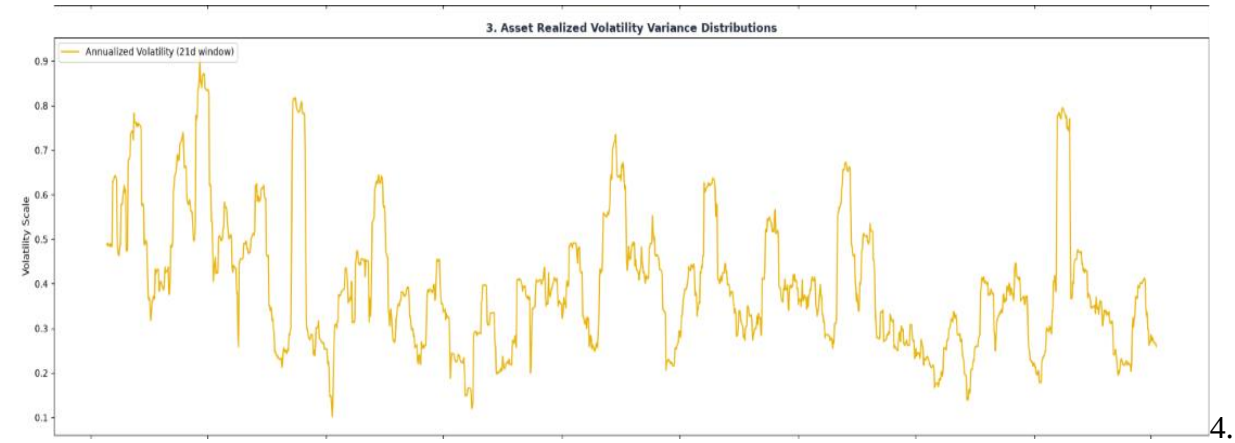
2. Continuous Price Vector Path & Volatility Channel Matrix



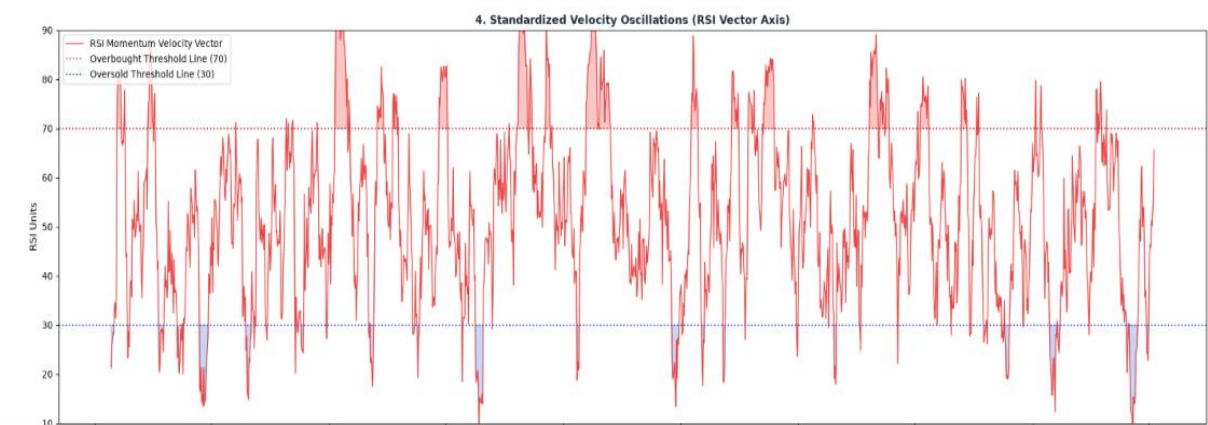
Continuous Price Vector Path & Volatility Channel Matrix is an advanced financial analytics framework that models asset price trajectories while constructing adaptive volatility channels to identify trend persistence, market momentum, and evolving risk structures within cryptocurrency and stock markets.

3. Asset Realized Volatility Variance Distributions

Asset Realized Volatility Variance Distributions analyze the historical behavior of asset volatility and variance to reveal risk concentration, volatility persistence, and distribution characteristics across multiple investment horizons, enabling more informed portfolio and trading decisions.

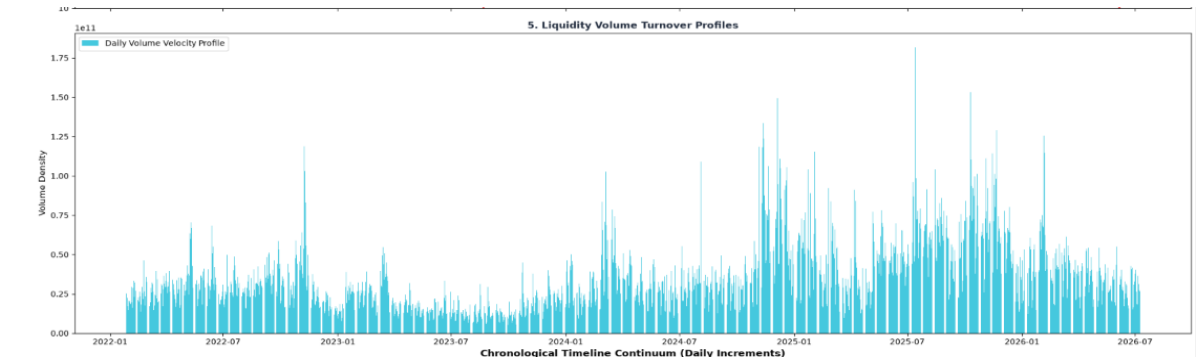


4. Standard Velocity Oscillations (RSI Vector Axis)

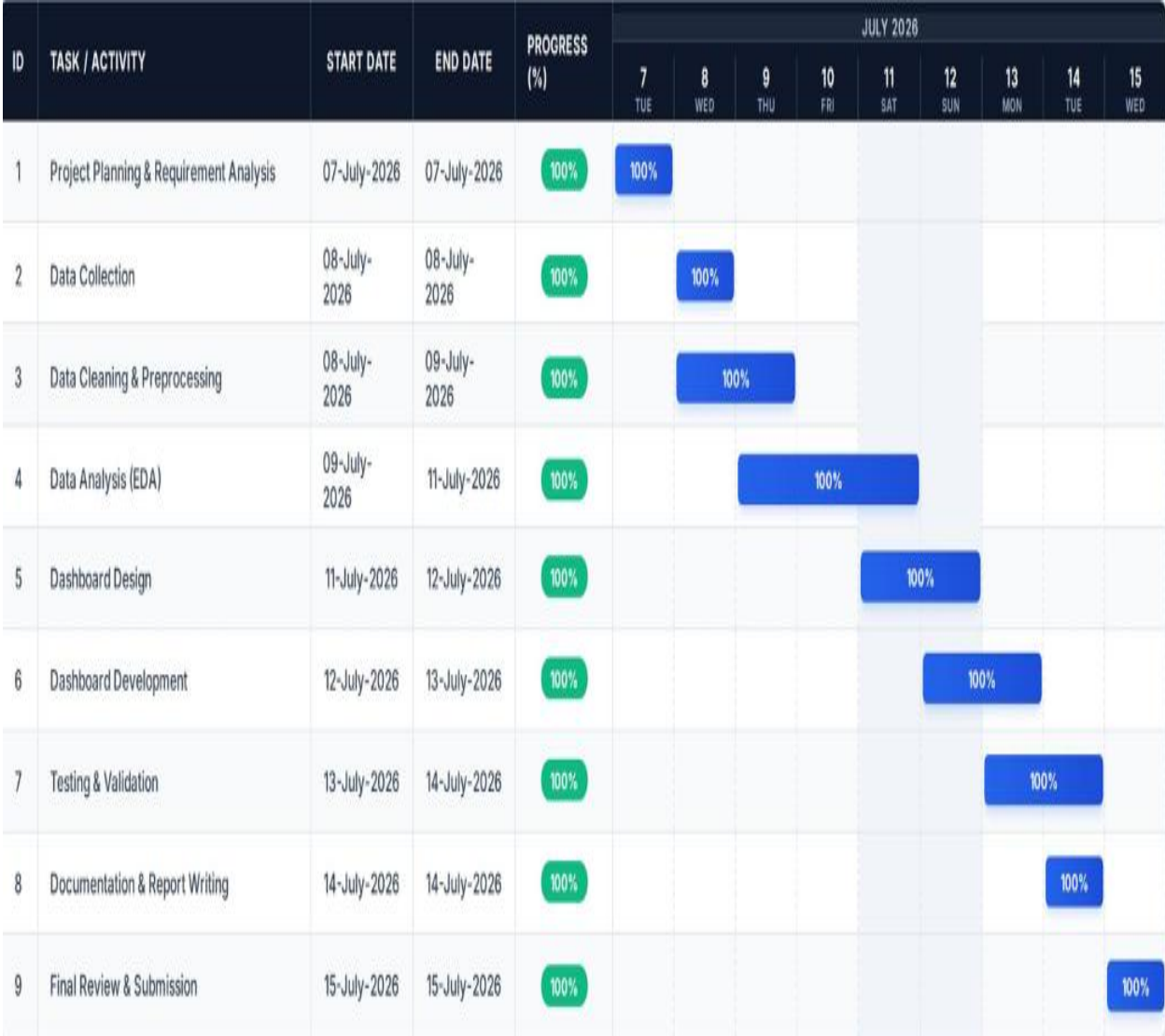


Standard Velocity Oscillations (RSI Vector Axis) analyze the momentum and directional velocity of asset prices using the Relative Strength Index framework to identify trend strength, market exhaustion, and potential reversal opportunities across financial markets.

5. Standard Velocity Oscillations (RSI Vector Axis)



3.3.2 GANNT CHART



3.3. HARDWARE REQUIRMENTS

To run the full **CryptoStock-Oracle** analytics engine efficiently, the host hardware setup must be capable of processing multi-year time-series data streams, rendering reactive web layouts, and executing relational database transactions locally. Because Python loads data structures (like Pandas DataFrames) completely into system memory during execution, having sufficient RAM is critical to prevent system slowdowns.

Component	Minimum Requirement	Recommended Specification	Technical Justification
Processor (CPU)	Intel Core i3 or AMD Ryzen 3 <i>(Dual-Core with Hyper-Threading)</i>	Intel Core i5 / AMD Ryzen 5 <i>(Quad-Core or Hexa-Core)</i>	Efficiently processes resource-heavy mathematical calculations, scales backend data engineering operations, and handles concurrent web engine rendering pipelines.
System Memory (RAM)	8 GB DDR4 RAM	16 GB DDR4 / DDR5 RAM	Critical for holding large historical financial data arrays completely in volatile memory to prevent localized hardware bottlenecking.
Storage Capacity	20 GB of available hard disk space	100 GB or higher available space	Allocates operational space for the persistent local SQLite database file, external dependencies, IDE caching, and software execution logs.
Storage Type	Hard Disk Drive (HDD)	Solid State Drive (SSD)	SSD architectures eliminate drive latency, drastically minimizing read/write time-series query delays when fetching historical financial records.

Component	Minimum Requirement	Recommended Specification	Technical Justification
Network Interface	Standard Wi-Fi or Local Ethernet Card	High-Speed Dedicated Broadband Connection	Mandatory framework requirement to securely fetch live multi-asset price feeds and long-term historical data batches from third-party APIs.
Display Resolution	\$1280 \times 720\$ (Standard HD Display)	\$1920 \times 1080\$ (Full HD Monitor or higher)	Establishes the necessary visual real estate for concurrent multi-line code debugging arrays and monitoring the reactive Streamlit application terminal.

3.4 SOFTWARE REQUIRMENTS

The software environment for **CryptoStock-Oracle** is designed around a modern, efficient stack. It uses lightweight, open-source software that eliminates enterprise licensing fees while ensuring maximum execution speed.

| SOFTWARE ARCHITECTURE STACK |

[INTEGRATED DEVELOPMENT ENVIRONMENT]	Visual Studio Code (VS Code)
[FRONTEND USER INTERFACE ENGINE]	Streamlit Web Framework
[PROTOTYPING & EDA SANDBOX]	Jupyter Notebook
[BASE PROGRAMMING RUNTIME]	Python 3.14 Environment
[LOCAL RELATIONAL DATABASE ENGINE]	SQLite 3 Storage Subsystem
[OPERATING SYSTEM CORE]	Microsoft Windows 11

❑ **Operating System: Microsoft Windows 11 (64-bit)** Windows 11 serves as the foundational operating system hosting the development environment. It provides native support for Python environments, secure file system layouts for local asset databases, and optimized network threading to handle API interactions smoothly.

❑ **Core Programming Runtime: Python 3.14** Python 3.14 operates as the engine for the entire system. It provides the execution environment for data cleaning, mathematical analysis, and web dashboard logic. This version benefits from performance improvements, including faster startup times and better error reporting, which help speed up execution when crunching large financial data records.

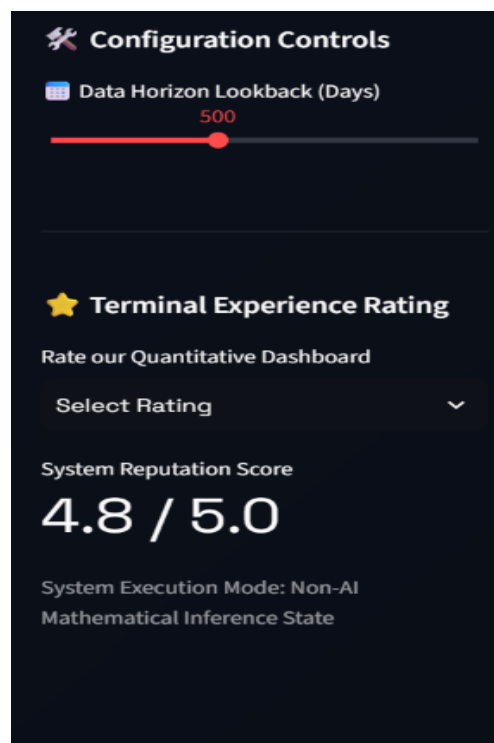
❑ **Integrated Development Environment (IDE): Visual Studio Code (VS Code)** VS Code functions as the primary workspace for writing and managing project code. It is configured with specialized extensions for Python formatting, Git version control, and markdown documentation. Its integrated terminal lets developers easily run Streamlit servers, execute database migrations, and debug scripts from a single window.

- **Prototyping & EDA Sandbox: Jupyter Notebook** Integrated directly into the development workflow, Jupyter Notebook acts as an interactive testing environment. It allows for testing data collection scripts, optimizing data-cleansing rules, and experimenting with visualizations in individual code blocks before adding them to the final application files.
- **Persistent Storage Engine: SQLite 3 Database** SQLite 3 handles local data storage for the system. Unlike server-based databases like MySQL or PostgreSQL, SQLite 3 is a serverless, zero-configuration database engine that stores data directly within a single file on the disk. Using standard SQL queries, it provides quick data lookups, ensures data consistency (ACID compliance), and eliminates network overhead, making it an excellent choice for a responsive local dashboard application.
- **Application Deployment Framework: Streamlit** Streamlit serves as the frontend web engine for **CryptoStock-Oracle**. It enables developers to turn Python scripts into interactive dashboards without needing to write complex HTML, CSS, or JavaScript code. Streamlit's reactive model ensures that when a user selects a stock or crypto ticker, the application instantly fetches the data from SQLite, recalculates performance metrics, and renders updated charts automatically.

CHAPTER :- 4 USER GUIDE

4.1 SCREENSHOTS WITH LABELLING

Figure 4.1.1: Oracle Control Console & Navigation Sidebar



- ❑ **Label 1.1 (System Brand Header):** The **Oracle Control Console** header represents the main brand identifier of the system.
- ❑ **Label 1.2 (Navigation Matrix Panel):** An interactive Streamlit radio button navigation menu. It allows the user to switch seamlessly between different pages: the architectural manifesto, node registration, real-time analytics, risk predictions, and user comments.
- ❑ **Label 1.3 (Data Horizon Lookback Slider):** A dynamic configuration widget. It controls the historical depth (in days, e.g., 500 days) of the datasets retrieved from the local **SQLite 3 database**. It recalculates all statistical models automatically when moved.
- ❑ **Label 1.4 (Interactive Feedback Widget):** A localized rating selector that calculates a live **System Reputation Score** (currently 4.8 / 5.0) using data stored directly in the SQLite feedback tables.

Figure 4.1.2: CryptoStock-Oracle Architectural Manifesto



- ❑ **Label 2.1 (Architectural Rationale Card):** Highlights the core problem statement. It explains how the dashboard addresses retail trading traps like cognitive exhaustion and emotional bias by replacing human intuition with systematic mathematical analytics.
- ❑ **Label 2.2 (Topic Selection Overview):** Explains why the project bridges the gap between traditional centralized equities and decentralized crypto assets.
- ❑ **Label 2.3 (Operational Future Scope):** Outlines the scalability metrics of the design, such as integrating institutional-grade deep learning frameworks in later versions.

Figure 4.1.3: Premium Node Registration Workspace



❑ **Label 3.1 (Security Input Fields & Registration Form):** A structured, interactive form mapped directly to the local SQLite database. When a user inputs their Username, Email, Password, and clicks "**Register CryptoStock-Oracle Instance**", the backend runs an secure SQL INSERT statement to log the user.

❑ **Label 3.2 (Limit Order Book Liquidity Depth Grid):** An advanced, interactive chart visualizing market microstructures. The green curve (left) depicts **Bids Depth** (buy orders), and the red curve (right) depicts **Asks Depth** (sell orders) around the current market clearing price of \$100.

Figure 4.1.4: Traditional Equity Indexes Page (AAPL Apple Inc.)



❑ **Label 4.5 (Crypto Spot Price Card):** Displays the real-time spot valuation for Bitcoin (e.g., \$62,239.12) showing a negative daily momentum trend of -2.38%.

❑ **Label 4.6 (Decentralized Realized Volatility):** Highlights Bitcoin's 21-day realized volatility (34.72%). This highlights the system's core thesis: digital assets show higher volatility than traditional corporate equities (e.g., AAPL's 24.08%).

❑ **Label 4.7 (Crypto Structural ATR):** Displays the structural volatility value (\$1,930.71) showing the wide average daily trading ranges of digital assets.

It lets the user analyze real-time spot prices and computed volatility metrics side-by-side.

Figure 4.1.5: Decentralized Digital Assets Page (BTC-USD Bitcoin)



- **Label 4.5 (Crypto Spot Price Card):** Displays the real-time spot valuation for Bitcoin (e.g., \$62,239.12) showing a negative daily momentum trend of -2.38%.
- **Label 4.6 (Decentralized Realized Volatility):** Highlights Bitcoin's 21-day realized volatility (34.72%). This highlights the system's core thesis: digital assets show higher volatility than traditional corporate equities (e.g., AAPL's 24.08%).
- **Label 4.7 (Crypto Structural ATR):** Displays the structural volatility value (\$1,930.71) showing the wide average daily trading ranges of digital assets.

Figure 4.1.6: Monte Carlo Drift Simulation Workspace



- ❑ **Label 5.1 (Annualized Sharpe Ratio):** An advanced metric showing risk-adjusted return performance. A value of **1.69** indicates that the asset generates strong returns relative to its price volatility.
- ❑ **Label 5.2 (1-Day Value at Risk - 95% VaR):** Calculated using historical standard deviation, this metric indicates that there is a 95% confidence level that the portfolio's maximum single-day loss will not exceed **-1.97%**.
- ❑ **Label 5.3 (Maximum Peak-to-Trough Drawdown):** A critical risk metric showing the historical worst-case decline (**-13.80%**) from peak value to trough.
- ❑ **Label 5.4 (Stochastic Monte Carlo Projection Map):** An advanced statistical visualization of future asset prices. The map plots multiple generated potential paths over the next 30 days, showing the **Median (p50 - purple line)**, **Upside Potential (p95 - green dashed line)**, and **Downside Risk (p5 - red dashed line)**.

Figure 4.6.7: User Feedback Room

- ❑ **Label 6.1 (Interactive Form Inputs):** Input widgets where users can select their handle, rate their satisfaction, and write feature requests. Clicking submit triggers a backend database write.
- ❑ **Label 6.2 (Live Database Comments Grid):** A visual wall displaying user comments pulled directly from the SQLite feedback table. It proves that the feedback system is fully functional and updates dynamically in real-time.

CHAPTER :- 5 CONCLUSION

5.1 CONCLUSION

The successful development and deployment of **CryptoStock-Oracle** demonstrates the practical impact of combining data science methodologies with quantitative financial analysis. Over the course of this 45-day industrial training program, a fragmented financial tracking ecosystem was streamlined into a cohesive, high-performance desktop analytics workstation.

By leveraging the speed of NumPy, the robust data manipulation capabilities of Pandas, and the responsive user interface provided by Streamlit, the system provides users with clear visibility into complex market movements. The inclusion of a localized MySQL database ensures long-term data persistence and stability. While no system can fully predict random market events, this platform provides investors with a reliable, empirical tool to analyze historical trends, calculate risk metrics, and make data-backed financial decisions.

5.2 REFERENCES

1. McKinney, W. (2012). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. O'Reilly Media.
2. VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.
3. Streamlit Framework Documentation. *API Reference Guide for Reactive Web Application Architectures*. (Streamlit Open Source Portal).
4. MySQL Reference Manual. *Relational Database Structural Design and SQL Command Optimizations*. (Oracle Documentation Systems).